BÁO CÁO THỰC HÀNH MINI PROJECT 02

LINUX WEB SERVER

Người báo cáo: Liamz07

1. Mục tiêu bài thực hành:

- Tạo tài khoản người dùng chuyên dụng và phân quyền quản trị an toàn trên Ubuntu Server.
- Tạo cặp khóa SSH và cấu hình xác thực bằng khóa công khai (Public Key Authentication) từ máy Client.
- Thực hiện các bước bảo mật nâng cao cho máy chủ (Hardening SSH, cấu hình tường lửa UFW).
- Cài đặt và cấu hình Web Server Nginx.
- Xây dựng thư mục ứng dụng, triển khai mã nguồn app.js và cấu hình Systemd Service chạy ngầm.
- Cấu hình Nginx làm Reverse Proxy để chuyển tiếp yêu cầu từ bên ngoài vào ứng dụng Node.js ở cổng nội bộ.

2. Thông tin môi trường thực hành:

- **Hệ điều hành Server:** Ubuntu Server (Địa chỉ IP: 192.168.10.132)
- **Hệ điều hành Client:** Kali Linux
- **Tài khoản thực hành:** deployer

3. Nội dung, phương pháp giải quyết chi tiết và kết quả:

PHẦN 1: TẠO TÀI KHOẢN QUẢN TRỊ DEPLOYER VÀ CẤP QUYỀN SUDO

- **Yêu cầu:** Tạo tài khoản người dùng mới tên là deployer và thêm vào nhóm quản trị viên sudo để thực hiện các thao tác cấu hình hệ thống an toàn thay cho tài khoản root mặc định.
- **Phương pháp giải quyết:** Sử dụng lệnh useradd để tạo tài khoản, thiết lập mật khẩu và thêm vào nhóm quyền sudo.
- **Lệnh thực thi & Cú pháp tổng quát:**
  - *Lệnh:*

```
sudo useradd -m -s /bin/bash deployer

sudo passwd deployer

sudo usermod -aG sudo deployer
```

  - *Cú pháp:*

```
sudo useradd [options] username

sudo passwd username

sudo usermod [options] groupname username
```

- *Giải thích câu lệnh:*
  - sudo useradd -m -s /bin/bash deployer
    - useradd: Công cụ dòng lệnh tiêu chuẩn trên Linux dùng để khởi tạo và thêm một tài khoản người dùng mới.
    - -m: Tùy chọn yêu cầu hệ thống tự động tạo thư mục chính (home directory) cho người dùng mới (thường là đường dẫn /home/deployer), nơi chứa cấu hình và dữ liệu riêng của họ.
    - -s /bin/bash: Tùy chọn chỉ định trình thông (shell) mặc định cho người dùng khi họ đăng nhập vào hệ thống là Bash (/bin/bash), cung cấp một môi trường dòng lệnh đầy đủ và phổ biến.
    - deployer: Tên tài khoản (username) của người dùng mới mà bạn muốn tạo ra.
  - sudo passwd deployer
    - passwd: Công cụ dòng lệnh chuẩn trên Linux dùng để quản lý và thay đổi mật khẩu (password) của tài khoản.
    - deployer: Tên tài khoản cụ thể muốn đổi mật khẩu.
  - sudo usermod -aG sudo deployer
    - usermod: Công cụ dòng lệnh dùng để chỉnh sửa thông tin hoặc thuộc tính của một tài khoản người dùng đã có sẵn.
    - -aG (--append, --groups): Tùy chọn kết hợp dùng để bổ sung người dùng vào một nhóm mới mà không làm mất các nhóm cũ.
    - sudo (từ thứ hai): Tên của nhóm đích mà bạn muốn đưa người dùng vào. Trên Ubuntu, nhóm sudo cấp quyền cho phép tài khoản đó thực hiện các lệnh quản trị hệ thống.
    - deployer: Tên tài khoản người dùng cụ thể đang được tác động (được đưa vào nhóm sudo).
- *Kết quả:* Chuyển sang tài khoản deployer và chạy thử lệnh sudo whoami, hệ thống trả về kết quả là root.
- *Giải thích kết quả:* Tài khoản deployer đã được cấp quyền quản trị thành công.

PHẦN 2: TẠO CẶP KHÓA SSH VÀ SAO CHÉP KHÓA LÊN SERVER

- **Yêu cầu:** Tạo cặp khóa mã hóa bất đối xứng (Public/Private Key) trên máy Client (Kali Linux) và sao chép khóa công khai lên Ubuntu Server để chuẩn bị cho việc xác thực không cần mật khẩu.
- **Phương pháp giải quyết:** Sử dụng công cụ ssh-keygen để tạo khóa và ssh-copy-id để truyền khóa công khai sang server.
- **Lệnh thực thi & Cú pháp tổng quát:**
  - *Lệnh:*

ssh-keygen -t rsa -b 4096

ssh-copy-id deployer@192.168.10.132

  - *Cú pháp:*

ssh-keygen [options]

ssh-copy-id [options] [user]@[hostname]

- *Giải thích câu lệnh:*
  - ssh-keygen -t rsa -b 4096
    - ssh-keygen: Công cụ dùng để tạo, quản lý và chuyển đổi các cặp khóa xác thực (public key và private key) cho giao thức SSH.
    - -t: Chỉ định thuật toán mã hóa khóa (Ví dụ: rsa, ed25519, ecdsa).
    - -b: Chỉ định độ dài của khóa tính bằng bit (Ví dụ: 4096 bit cho RSA giúp tăng cường bảo mật).
  - ssh-copy-id deployer@192.168.10.132
    - ssh-copy-id: Công cụ dùng để tự động sao chép khóa công khai (public key) lên máy chủ từ xa và thêm nó vào tệp ~/.ssh/authorized_keys ở đó.
    - deployer@192.168.10.132: Tên tài khoản và địa chỉ IP (hoặc tên miền) của máy chủ đích mà bạn muốn cấp quyền truy cập.
- *Kết quả:* Khóa được tạo thành công trong thư mục ~/.ssh/, lệnh ssh-copy-id hoàn tất việc truyền khóa mà không báo lỗi.
- *Giải thích kết quả:* Máy Client đã sở hữu chứng thực mã hóa để kết nối an toàn vào Server.

PHẦN 3: BẢO MẬT DỊCH VỤ SSH (HARDENING SSH & GIỚI HẠN ALLOWUSERS)

- **Yêu cầu:** Vô hiệu hóa hoàn toàn phương thức đăng nhập bằng mật khẩu (PasswordAuthentication no), bắt buộc sử dụng khóa công khai và chỉ định rõ tài khoản nào được phép kết nối qua SSH (AllowUsers).
- **Phương pháp giải quyết:** Chỉnh sửa tệp cấu hình chính của SSH (/etc/ssh/sshd_config), cấu hình tắt mật khẩu, thêm dòng giới hạn AllowUsers deployer và kiểm tra các tệp ghi đè trong sshd_config.d/.
- **Lệnh thực thi & Cú pháp tổng quát:**
  - *Lệnh:*

sudo nano /etc/ssh/sshd_config

sudo ls -la /etc/ssh/sshd_config.d/

sudo systemctl restart ssh

  - *Cú pháp:*

sudo [text_editor] [file_path]

sudo ls [options] [directory_path]

sudo systemctl [actions] [service_name]

  - *Giải thích câu lệnh:*
    - sudo nano /etc/ssh/sshd_config
      - nano: Trình soạn thảo văn bản đơn giản và trực quan ngay trên giao diện dòng lệnh của Linux.
      - /etc/ssh/sshd_config: Đường dẫn tuyệt đối đến tệp cấu hình chính của dịch vụ SSH Server trên hệ thống.
    - sudo ls -la /etc/ssh/sshd_config.d/
      - ls: Lệnh dùng để liệt kê danh sách các tệp tin và thư mục.

- -l (long): Hiển thị dưới dạng danh sách chi tiết (bao gồm quyền hạn, chủ sở hữu, kích thước, và thời gian chỉnh sửa).
- -a (all): Hiển thị toàn bộ các tệp tin, bao gồm cả các tệp ẩn (tệp bắt đầu bằng dấu chấm .).
- /etc/ssh/sshd_config.d/: Thư mục chứa các tệp cấu hình SSH bổ sung (cho phép chia nhỏ cấu hình thành các tệp riêng biệt thay vì dồn tất cả vào tệp chính).

❖ sudo systemctl restart ssh

- systemctl: Công cụ chính dùng để điều khiển và quản lý các dịch vụ (services) thông qua trình quản lý hệ thống systemd.
- restart: Hành động yêu cầu khởi động lại dịch vụ (tương đương với việc dừng hẳn dịch vụ rồi khởi động lại ngay lập tức).
- ssh: Tên định danh của dịch vụ SSH trên hệ thống (trên một số bản phân phối Linux khác, tên này có thể là sshd).

o *Cấu hình bên trong file sshd_config:*

PasswordAuthentication no

KbdInteractiveAuthentication no

AllowUsers deployer

o *Ý nghĩa các câu lệnh:*

❖ Tắt tính năng xác thực bằng mật khẩu truyền thống và xác thực bàn phím tương tác để ngăn chặn hoàn toàn tấn công Brute-force.

❖ Tùy chọn AllowUsers deployer ép buộc dịch vụ SSH chỉ chấp nhận kết nối từ duy nhất tài khoản deployer, từ chối tất cả các user khác trên hệ thống dù có đúng thông tin đăng nhập, giúp cô lập và bảo mật vùng quản trị.

o *Kết quả:* Kiểm tra từ máy Kali Linux bằng lệnh ssh -o PubkeyAuthentication=no deployer@192.168.10.131, hệ thống từ chối và báo lỗi Permission denied (publickey).

o *Giải thích kết quả:* Dịch vụ SSH đã được bảo mật tuyệt đối, chỉ chấp nhận kết nối từ các client có khóa riêng (private key) hợp lệ thuộc đúng user được phép.

PHẦN 4: CÀI ĐẶT WEB SERVER NGINX VÀ CẤU HÌNH UFW

- **Yêu cầu:** Cài đặt Nginx và cấu hình tường lửa UFW để mở các cổng dịch vụ an toàn.
- **Phương pháp giải quyết:** Cập nhật kho gói tin, cài đặt Nginx qua apt và thiết lập quy tắc cho phép OpenSSH cùng Nginx Full trên UFW.
- **Lệnh thực thi & Cú pháp tổng quát:**
  - *Lệnh*:

sudo apt update && sudo apt install nginx -y

sudo ufw allow OpenSSH

sudo ufw allow 'Nginx Full'

sudo ufw enable

  - *Cú pháp*:

sudo apt update && sudo apt install <package_name> [options]

sudo ufw allow <service_name_or_port>

sudo ufw allow '<profile_name>'

sudo ufw <action>

- *Giải thích câu lệnh*:
  - sudo apt update && sudo apt install nginx -y
    - apt: Trình quản lý gói phần mềm chính trên các hệ điều hành dựa trên Debian/Ubuntu.
    - update: Lệnh con của apt dùng để đồng bộ và làm mới danh sách các gói phần mềm mới nhất từ kho lưu trữ (repository).
    - &&: Toán tử logic "AND", đảm bảo lệnh phía sau chỉ được thực thi khi lệnh phía trước chạy thành công.
    - install: Lệnh con của apt dùng để tiến hành cài đặt một gói phần mềm mới.
    - nginx: Tên gói phần mềm cụ thể muốn cài đặt (máy chủ web Nginx).
    - -y: Tùy chọn tự động trả lời "yes" (đồng ý) cho các thông báo xác nhận dung lượng hoặc tiếp tục cài đặt, giúp câu lệnh chạy hoàn toàn tự động mà không cần người dùng nhập tay.
  - sudo ufw allow OpenSSH
    - ufw: Viết tắt của Uncomplicated Firewall, công cụ quản lý tường lửa đơn giản và phổ biến trên Ubuntu.
    - allow: Hành động ra lệnh cho tường lửa mở cổng hoặc cho phép một loại lưu lượng truy cập đi qua.
    - OpenSSH: Hồ sơ (profile) được định nghĩa sẵn trong hệ thống cho dịch vụ SSH, giúp mở cổng 22 để bạn không bị mất kết nối từ xa khi bật tường lửa.
  - sudo ufw allow 'Nginx Full'
    - ufw: Công cụ quản lý tường lửa.
    - allow: Hành động cho phép kết nối.
    - 'Nginx Full': Hồ sơ tích hợp sẵn của UFW dành cho Nginx, cho phép đồng thời cả hai lưu lượng: HTTP (cổng 80) và HTTPS (cổng 443).
  - sudo ufw enable
    - ufw: Công cụ quản lý tường lửa.
    - enable: Hành động kích hoạt tường lửa chính thức đi vào hoạt động, bắt đầu áp dụng các quy tắc (rules) bạn đã cấu hình trước đó (như cho phép SSH và Nginx) và chặn các kết nối mặc định khác.
- *Ý nghĩa các câu lệnh:* Cài đặt Nginx và kích hoạt tường lửa UFW chỉ cho phép các cổng kết nối cần thiết hoạt động.
- *Kết quả:* Nginx hoạt động ở trạng thái active (running), tường lửa UFW kích hoạt ổn định.
- *Giải thích kết quả:* Máy chủ sẵn sàng tiếp nhận kết nối HTTP/HTTPS từ bên ngoài và đảm bảo an toàn quản trị từ xa qua SSH.

PHẦN 5: TRIỂN KHAI MÃ NGUỒN ỨNG DỤNG NODE.JS (APP.JS)

- **Đề bài:** Tạo thư mục chứa mã nguồn ứng dụng web và viết tệp ứng dụng Node.js (app.js) lắng nghe kết nối ở cổng nội bộ 3000.
- **Phương pháp giải quyết:** Tạo thư mục /var/www/myapp, phân quyền sở hữu cho user deployer, sau đó tạo và biên tập tệp app.js.
- **Lệnh thực thi & Cú pháp tổng quát:**
  - *Lệnh:*

```
sudo mkdir -p /var/www/myapp

sudo chown -R $USER:$USER /var/www/myapp

cd /var/www/myapp

nano app.js
```

  - *Cú pháp:*

```
sudo mkdir [options] directory

sudo chown [options] user:group directory

cd [directory]

nano [file]
```

  - *Giải thích câu lệnh:*
    - sudo mkdir -p /var/www/myapp
      - mkdir: Lệnh viết tắt của Make Directory, dùng để tạo thư mục mới trên hệ thống Linux.
      - -p: Tùy chọn viết tắt của --parents, cho phép tạo đồng thời cả các thư mục cha nếu chúng chưa tồn tại (Ví dụ: Tạo cả /var và /var/www trước khi tạo myapp) và không báo lỗi nếu thư mục đã tồn tại.
      - /var/www/myapp: Đường dẫn tuyệt đối đến thư mục cần tạo (myapp nằm trong /var/www/).
    - sudo chown -R $USER:$USER /var/www/myapp
      - chown: Viết tắt của Change Owner, lệnh dùng để thay đổi chủ sở hữu (owner) và nhóm sở hữu (group) của tệp hoặc thư mục.
      - -R: Tùy chọn viết tắt của --recursive (đệ quy), giúp áp dụng thay đổi quyền sở hữu cho toàn bộ các tệp và thư mục con nằm bên trong đường dẫn.
      - $USER:$USER: Chỉ định chủ sở hữu mới và nhóm mới. Biến hệ thống $USER tự động lấy tên của tài khoản bạn đang đăng nhập hiện tại. Cú pháp user:group đặt cả chủ sở hữu và nhóm quản lý đều là bạn.
      - /var/www/myapp: Đường dẫn tới thư mục đích cần thay đổi quyền sở hữu.
    - cd /var/www/myapp
      - cd: Viết tắt của Change Directory, lệnh dùng để di chuyển qua lại giữa các thư mục trên dòng lệnh.
      - /var/www/myapp: Đường dẫn của thư mục đích mà bạn muốn chuyển đến (trong trường hợp này là di chuyển vào bên trong thư mục myapp vừa tạo).
    - nano app.js

- nano: Trình soạn thảo văn bản đơn giản và trực quan ngay trên giao diện dòng lệnh của Linux.
- app.js: Tên tệp tin cần mở để chỉnh sửa. Nếu tệp app.js chưa tồn tại trong thư mục, nano sẽ tự động tạo mới một tệp trống có tên đó khi bạn lưu lại.

- *Mã nguồn bên trong app.js:*

```
const http = require('http');
const hostname = '0.0.0.0';
const port = 3000;

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello from Node.js on Ubuntu Server!\n');
});

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

- *Ý nghĩa các câu lệnh:* Khởi tạo thư mục chuẩn cho ứng dụng web, cấp quyền sở hữu an toàn và tạo máy chủ HTTP đơn giản bằng Node.js chạy trên cổng 3000.
- *Kết quả:* Kiểm tra bằng lệnh curl http://localhost:3000 trên server nhận được phản hồi chính xác từ ứng dụng.
- *Giải thích kết quả:* Ứng dụng Node.js đã sẵn sàng triển khai dưới dạng dịch vụ nền của hệ thống.

PHẦN 6: TẠO SYSTEMD SERVICE CHO NODE.JS

- **Yêu cầu:** Cấu hình ứng dụng Node.js chạy ngầm như một tiến trình hệ thống (daemon) để tự động khởi động cùng hệ thống.
- **Phương pháp giải quyết:** Tạo tệp dịch vụ myapp.service trong thư mục /etc/systemd/system/.
- **Lệnh thực thi & Cú pháp tổng quát:**
  - *Lệnh:*

sudo nano /etc/systemd/system/myapp.service

sudo systemctl daemon-reload

sudo systemctl start myapp

sudo systemctl enable myapp

  - *Cú pháp:*

sudo [text_editor] [file_path]

sudo systemctl daemon-reload

sudo systemctl start [service_name]

sudo systemctl enable [service_name]

- *Giải thích câu lệnh:*
  - sudo nano /etc/systemd/system/myapp.service
    - nano: Trình soạn thảo văn bản đơn giản và trực quan ngay trên giao diện dòng lệnh của Linux.
    - /etc/systemd/system/myapp.service: Đường dẫn tuyệt đối đến tệp định nghĩa dịch vụ (unit file) của systemd dành riêng cho ứng dụng myapp.
  - sudo systemctl daemon-reload
    - systemctl: Công cụ chính dùng để điều khiển và quản lý các dịch vụ thông qua trình quản lý hệ thống systemd.
    - daemon-reload: Lệnh yêu cầu tiến trình quản lý hệ thống quét lại và nạp (load) các tệp cấu hình dịch vụ mới được tạo ra hoặc vừa chỉnh sửa trên đĩa cứng, giúp hệ thống nhận diện được cấu hình cập nhật.
  - sudo systemctl start myapp
    - systemctl: Công cụ quản lý dịch vụ hệ thống.
    - start: Hành động yêu cầu khởi động dịch vụ ngay lập tức.
    - myapp: Tên định danh của dịch vụ cần khởi chạy (tương ứng với tệp myapp.service vừa tạo).
  - sudo systemctl enable myapp
    - systemctl: Công cụ quản lý dịch vụ hệ thống.
    - enable: Hành động kích hoạt dịch vụ để nó tự động chạy nền mỗi khi hệ điều hành khởi động (boot).
    - myapp: Tên dịch vụ được cấu hình để khởi động cùng hệ thống.
- *Nội dung file cấu hình:*

```
[Unit]
Description=Node.js Application Service
After=network.target

[Service]
Type=simple
User=deployer
WorkingDirectory=/var/www/myapp
ExecStart=/usr/bin/node /var/www/myapp/app.js
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

- *Ý nghĩa các câu lệnh:* Khai báo dịch vụ hệ thống Linux để quản lý vòng đời ứng dụng Node.js, chạy dưới tài khoản deployer và tự động khởi động khi reboot server.
- *Kết quả:* Lệnh sudo systemctl status myapp hiển thị trạng thái active (running).

- *Giải thích kết quả:* Ứng dụng Node.js hoạt động bền vững dưới dạng tiến trình nền (daemon).

PHẦN 7: CẤU HÌNH NGINX LÀM REVERSE PROXY

- **Yêu cầu:** Định tuyến các yêu cầu gửi đến Nginx (cổng 80) chuyển tiếp trực tiếp vào ứng dụng Node.js ở cổng nội bộ 3000.
- **Phương pháp giải quyết:** Chỉnh sửa tệp cấu hình mặc định của Nginx, thay thế khối xử lý tệp tĩnh bằng lệnh proxy_pass.
- **Lệnh thực thi & Cú pháp:**
  - *Lệnh:*

sudo nano /etc/nginx/sites-available/default

sudo nginx -t

sudo systemctl restart nginx

  - *Cú pháp:*

sudo [text_editor] [file_path]

sudo nginx [options]

sudo systemctl [action] [service_name]

  - *Giải thích câu lệnh:*
    - sudo nano /etc/nginx/sites-available/default
      - /etc/nginx/sites-available/default: Đường dẫn tuyệt đối đến tệp cấu hình máy chủ ảo (virtual host) mặc định của Nginx trên hệ thống.
    - sudo nginx -t
      - nginx: Tên chương trình thực thi (executable binary) của máy chủ web Nginx.
      - -t: Tùy chọn viết tắt của test configuration, có chức năng yêu cầu Nginx kiểm tra toàn bộ cú pháp trong các tệp cấu hình xem có bị lỗi (syntax error) hay không trước khi áp dụng thực tế.
  - *Nội dung file cấu hình:*

```
location / {
    proxy_pass http://localhost:3000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection 'upgrade';
    proxy_set_header Host $host;
    proxy_cache_bypass $http_upgrade;
}
```

  - *Giải thích file cấu hình:*
    - location / {: Khai báo khối cấu hình áp dụng cho tất cả các yêu cầu truy cập vào đường dẫn gốc (/) và các nhánh con của trang web.

- ❖ proxy_pass http://localhost:3000;: Chuyển tiếp toàn bộ các yêu cầu (requests) nhận được từ người dùng tới ứng dụng nội bộ đang chạy trên máy chủ tại cổng 3000 (Ví dụ: Ứng dụng Node.js, Express, hoặc React SSR).
- ❖ proxy_http_version 1.1;: Buộc Nginx sử dụng giao thức HTTP/1.1 khi giao tiếp với ứng dụng backend. Phiên bản này là bắt buộc nếu muốn duy trì kết nối liên tục (keep-alive) hoặc sử dụng WebSocket.
- ❖ proxy_set_header Upgrade $http_upgrade;: Chuyển tiếp tiêu đề Upgrade từ trình duyệt của người dùng xuống ứng dụng backend để cho phép nâng cấp giao thức kết nối khi cần thiết.
- ❖ proxy_set_header Connection 'upgrade';: Đặt tiêu đề Connection thành upgrade, thông báo rõ cho backend rằng client muốn chuyển đổi kết nối hiện tại sang dạng kết nối thời gian thực (WebSocket).
- ❖ proxy_set_header Host $host;: Giữ nguyên tên miền hoặc địa chỉ IP gốc mà người dùng đang gõ trên trình duyệt (thay vì đổi thành localhost:3000) để ứng dụng backend nhận diện đúng domain.
- ❖ proxy_cache_bypass $http_upgrade;: Yêu cầu hệ thống bỏ qua bộ nhớ đệm (cache) khi có yêu cầu nâng cấp kết nối, giúp các kết nối như WebSocket hoạt động mượt mà không bị ảnh hưởng bởi dữ liệu cũ trong cache.
- ❖ }: Dấu đóng khối cấu hình của location.

- *Ý nghĩa các câu lệnh:* Chuyển tiếp toàn bộ HTTP request nhận được từ Nginx sang ứng dụng Node.js phía sau, đồng thời duy trì các tiêu đề kết nối WebSocket và phiên làm việc.
- *Kết quả: L*ệnh nginx -t báo cú pháp thành công (syntax is ok), khi dùng lệnh curl http://192.168.10.132 từ máy Kali Linux nhận được phản hồi từ ứng dụng Node.js.
- *Giải thích kết quả:* Nginx đóng vai trò là cổng giao tiếp trung gian (Reverse Proxy) phân phối lưu lượng truy cập mạng hiệu quả và bảo mật cho ứng dụng.

4. Những lỗi có thể phát sinh trong lúc thực hành và cách khắc phục:

- **Lỗi không cho phép sao chép khóa (Permission denied) mặc dù đã nhập đúng mật khẩu của user:**

Nguyên nhân:

1) Do tùy chọn PasswordAuthentication trong tệp cấu hình /etc/ssh/sshd_config trên server đang bị cấu hình là no trước khi thực hiện sao chép khóa, khiến dịch vụ SSH từ chối mọi phương thức xác thực bằng mật khẩu (kể cả lệnh ssh-copy-id).
2) Do quyền hạn (permissions) của thư mục ~/.ssh hoặc tệp authorized_keys trên phía server không đạt chuẩn bảo mật yêu cầu.

Cách khắc phục:

1) Mở tệp cấu hình SSH trên server (sudo nano /etc/ssh/sshd_config), tạm thời đổi PasswordAuthentication no thành yes, sau đó khởi động lại dịch vụ (sudo systemctl restart ssh) và reboot lại Ubuntu Server. Tiến hành chạy lại lệnh ssh-copy-id từ máy client, nhập mật khẩu thành công rồi mới cấu hình lại thành no.
2) Kiểm tra và phân quyền chính xác trên server bằng các lệnh:

chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys

- **Lỗi SSH vẫn cho phép đăng nhập bằng mật khẩu dù đã cấu hình no ở tệp chính:**

Nguyên nhân: Do cơ chế Include của OpenSSH nạp các tệp cấu hình con trong thư mục /etc/ssh/sshd_config.d/ sau tệp chính, dẫn đến việc bị ghi đè giá trị yes.

Cách khắc phục: Kiểm tra các tệp .conf trong thư mục sshd_config.d/, chỉnh sửa tùy chọn PasswordAuthentication thành no tại các tệp đó rồi khởi động lại dịch vụ SSH.

- **Lỗi định dạng khi sử dụng phím Tab trong trình soạn thảo nano:**

Nguyên nhân: Phím Tab tạo ra ký tự tab cứng có thể gây cảnh báo hoặc lỗi cú pháp trong tệp cấu hình Nginx.

Cách khắc phục: Xóa các ký tự tab cứng và sử dụng phím Space để căn lề.

- **Lỗi thiếu tệp khi khởi chạy ứng dụng (Script not found):**

Nguyên nhân: Chưa tạo tệp mã nguồn app.js trong thư mục làm việc /var/www/myapp hoặc đặt sai đường dẫn thực thi.

Cách khắc phục: Kiểm tra kỹ thư mục hiện tại bằng lệnh ls -la và đảm bảo tệp app.js tồn tại đúng đường dẫn đã khai báo.

5. Kết luận và bài học rút ra:

- Nắm vững kỹ thuật tạo tài khoản quản trị chuyên dụng và phân quyền an toàn trên Linux.
- Thành thạo kỹ năng tạo cặp khóa SSH (ssh-keygen), phân phối khóa công khai (ssh-copy-id) và cấu hình giới hạn định danh truy cập chuyên sâu với AllowUsers.
- Hiểu rõ cách xử lý các xung đột phân quyền và cấu hình xác thực SSH trong quá trình triển khai thực tế.
- Hoàn thành thành công toàn bộ quy trình thiết lập môi trường máy chủ chuẩn sản xuất (Production) trên Ubuntu Server.
- Nắm vững kỹ thuật bảo mật cấu hình SSH nâng cao và vận hành tường lửa UFW hiệu quả.
- Hiểu rõ cơ chế triển khai mã nguồn Node.js, cấu hình Systemd Service kết hợp với Nginx Reverse Proxy giúp ứng dụng chạy bền vững, bảo mật và hiệu năng cao.

HẾT!